ỨNG DỤNG CỦA PHƯƠNG PHÁP MÔ PHỎNG MONTE CARLO VỚI XÁC SUẤT HÌNH HỌC

Nguyễn Cao Cường
Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
Thành phố Hồ Chí Minh, Việt Nam
23520198@gm.uit.edu.vn

Nguyễn Hữu Việt Cường
Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
Thành phố Hồ Chí Minh, Việt Nam
23520203@gm.uit.edu.vn

Nguyễn Quốc Đạt
Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
Thành phố Hồ Chí Minh, Việt Nam
23520260@gm.uit.edu.vn

***Tóm tắt* — Trong bài tiểu luận này, chúng tôi thực hiện trình bày lại các nghiên cứu của tác giả Nguyễn Thị Phi Doan [1] trong việc sử dụng mô phỏng Monte Carlo để tính xấp xỉ xác suất một biến cố dựa vào định nghĩa xác suất hình học bằng ngôn ngữ Python. Từ đó, tác giả đưa ra phương pháp xấp xỉ một số tích phân và các số siêu việt.**

Keywords — xác suất hình học, Monte Carlo

I. GIỚI THIỆU

Trong khoa học và phân tích số liệu, để kiểm chứng một đề xuất hoặc lý thuyết, bên cạnh các chứng minh toán học, người ta thường sử dụng phương pháp mô phỏng để minh họa và kiểm chứng kết quả. Đặc biệt với các quá trình phức tạp hoặc bài toán khó giải quyết bằng công thức giải tích (như tính tích phân hàm không có nguyên hàm sơ cấp), phương pháp mô phỏng trở nên rất hữu hiệu.

Bằng cách sử dụng phương pháp mô phỏng Monte Carlo kết hợp với định nghĩa xác suất hình học để tính xấp xỉ xác suất của các biến cố, từ đó ta có thể tính xấp xỉ giá trị của các tích phân xác định và các số siêu việt như số π.

II. CƠ SỞ LÝ THUYẾT

A. Mô phỏng Monte Carlo

Mô phỏng Monte Carlo là kỹ thuật mô phỏng xác suất dựa trên lặp lại nhiều lần các kịch bản ngẫu nhiên nhằm đánh giá tác động của rủi ro và sự không chắc chắn lên tiến độ, chi phí hoặc kết quả dự án. Phương pháp mô phỏng này dựa trên hai luật quan trọng của xác suất là luật số lớn mạnh và luật số lớn yếu.

Mô phỏng Monte Carlo bao gồm ba bước cơ bản:

- Thiết lập mô hình dự đoán: xác định biến phụ thuộc cần dự đoán và các biến độc lập/thuộc tính.
- Xác định phân phối xác suất của các biến độc lập: Sử dụng dữ liệu lịch sử và hoặc đánh giá chủ quan của nhà phân tích để xác định một loạt các giá trị có thể xảy ra và chỉ định trọng số xác suất cho từng giá trị.
- Chạy mô phỏng lặp đi lặp lại, tạo ra các giá trị ngẫu nhiên của các biến độc lập. Làm điều này cho đến khi thu thập đủ kết quả để tạo thành một mẫu đại diện cho số lượng gần như vô hạn các kết hợp có thể có.

B. Luật số lớn yếu

<u>Định lý:</u> Cho $X_1,...,X_N$ là một tập hợp các biến ngẫu nhiên độc lập và cùng phân phối có kỳ vọng μ và phương sai σ^2. Giả sử $\mathbb{E}\left[X^2\right]<\infty$. Đặt $\overline{X_N}=\frac{1}{N}\sum_{n=1}^{N}X_n$, khi đó với mọi $\varepsilon>0$:

$$\lim_{N\to\infty}\mathbb{P}\left[\left|\overline{X_N}-\mu\right|>\varepsilon\right]=0 \qquad (1)$$

Định lý (1) trình bày về sự hội tụ của một dãy các giá trị xác xuất. Mặc dù xác suất để sai số xảy ra sẽ tiến về 0 khi tăng lên, nó không loại trừ hoàn toàn khả năng sai số vẫn có thể xảy ra ở một thời điểm nào đó.

C. Luật số lớn mạnh

<u>Định lý:</u> Cho $X_1,...,X_N$ là một tập hợp các biến ngẫu nhiên độc lập và cùng phân phối có kỳ vọng μ và phương sai σ^2. Giả sử $\mathbb{E}\left[X^4\right]<\infty$. Đặt $\overline{X}_N=\frac{1}{N}\sum_{n=1}^{N}X_n$, X_n là trung bình mẫu, khi đó:

$$\mathbb{P}\left[\lim_{N\to\infty}\overline{X_N}=\mu\right]=1. \qquad (2)$$

Định lý (2) trình bày về sự hội tụ của chính biến ngẫu nhiên. Đối tượng giới hạn của trung bình mẫu chính xác là hằng số μ. Số lần sai lệch là hữu hạn. Sau khi qua hết số lỗi hữu hạn này, ta sẽ hoàn toàn không còn gặp lỗi nữa.

D. Xác suất hình học

Giả sử phép thử có các biến cố sơ cấp đồng khả năng, mỗi biến cố sơ cấp được biểu diễn bởi một điểm hình học trong miền H nào đó, biến cố B được biểu diễn bởi miền B là một phần của miền H. Khi đó, xác suất của B được định nghĩa:

$$P(B)=\frac{m(B)}{m(H)} \qquad (3)$$

Trong đó, $m(B)$ và $m(H)$ là độ đo hình học của miền B và H tương ứng. Độ đo hình học có thể là độ dài, diện tích và thể tích (lần lượt tương ứng trong không gian một chiều, hai chiều và ba chiều).

Từ định nghĩa (3), nếu ta biết được độ đo $m(H)$ của miền H và ước lượng xác suất $P(B)$ thông qua mô phỏng Monte Carlo, ta có thể xác định được độ đo $m(B)$ của miền B.

III. PHƯƠNG PHÁP THỰC HIỆN VÀ KẾT QUẢ

A. Thiết kế mô hình mô phỏng

Dựa trên các bài toán được đưa ra của tác giả Nguyễn Thị Phi Đoan [1], chúng tôi thực hiện mô phỏng lại các ví dụ bằng ngôn ngữ Python trên môi trường Google Colab với các công cụ hỗ trợ:

- Thư viện Numpy: tạo sinh dữ liệu ngẫu nhiên tuân theo luật phân phối đều.
- Thư viện Matplotlib: trực quan hoá dữ liệu.

Quy trình thuật toán mô phỏng Monte Carlo được trình bày qua các bước như sau:

- ***Bước 1:*** Xác định miền cơ sở H bằng cách chọn một miền hình học đảm bảo tính chất xác định được độ đo $m(H)$ một cách dễ dàng và bao trùm lên đối tượng nghiên cứu.
- ***Bước 2:*** Xác định điều kiện của biến cố B bằng cách mô hình hoá các biến cố cần tính toán thành tập hợp các bất đẳng thức. Các điểm thoả mãn các bất đẳng thức này được xem là thuộc về miền B.
- ***Bước 3:*** Tạo sinh ngẫu nhiên một lượng lớn n_H các điểm dữ liệu ngẫu nhiên trải đều trong miền H. Toạ độ (x, y) của các điểm đảm bảo tuân thủ luật phân phối đều để đảm bảo tính khách quan của xác suất.
- ***Bước 4:*** Duyệt qua toàn bộ n_H điểm đã sinh ra để kiểm tra toạ độ các điểm có thoả mãn điều kiện của miền B đã trình bày tại bước 2 hay không. Từ đó, ta xác định được n_B điểm thoả mãn.
- ***Bước 5:*** Dựa trên định nghĩa xác suất hình học và luật số lớn, giá trị cần tìm được tính xấp xỉ bằng công thức $\frac{m(B)}{m(H)} = \frac{n_B}{n_H} \Leftrightarrow m(B) = m(H) \times \frac{n_B}{n_H}$

B. Thực hiện giải quyết các bài toán

1) Bài toán 1

Giả sử ta cần tính xấp xỉ tích tích phân hàm Laplace (phân phối chuẩn) của tích phân sau:

$$I = \frac{1}{\sqrt{2\pi}} \int_0^2 e^{-\frac{x^2}{2}} dx$$

Trước tiên, ta chọn miền H là miền hình chữ nhật sao cho

$$H = \{(x, y) : 0 \le x \le 2, 0 \le y \le 1\} \quad (4)$$

Ta xác định được độ đo $m(H)$ là diện tích của hình chữ nhật, tức $m(H) = 2 \times 1 = 2$.

Miền B là tập hợp các điểm nằm bên dưới đường cong của hàm mật độ cũng chính là diện tích I, thoả mãn điều kiện:

$$B = \left\{(x, y) \in H : y \le \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}\right\} \quad (5)$$

Đoạn chương trình sau thực hiện tạo sinh $n_H = 10000$ điểm toạ độ (x, y) đảm bảo tuân thủ luật phân phối đều với $x \in [0, 2]$ và $y \in [0, 1]$:

Bảng 1. Hàm thực hiện tạo sinh n_H điểm (x, y) thoả mãn $x \in [0, 2]$ và $y \in [0, 1]$

```
def run_single_experiment(n_samples=10000):
    x = np.random.uniform(0, 2, n_samples)
    y = np.random.uniform(0, 1, n_samples)
    curve_y = (1 / np.sqrt(2 * np.pi)) * np.exp(-
x**2 / 2)
    condition = y < curve_y
    n_B = np.sum(condition)
    return n_B, x, y, condition
```

Ta xác định được $n_B = 2376$, tức là có 2376 trên 10000 điểm n_H thoả điều kiện (5). Do đó ta xác định được giá trị xấp xỉ độ đo $m(H)$ hoặc giá trị xấp xỉ của tích phân I là:

$$m(B) = m(H) \times \frac{n_B}{n_H} = 2 \times \frac{2376}{10000} = 0.4764$$

với sai số $\Delta m(B) = |0.4772 - m(B)| = |0.4772 - 0.4764| = 0.0008$

Để hạn chế sai số, ta thực hiện lặp lại 10000 lần và lấy giá trị trung bình của các mẫu bằng đoạn chương trình sau:

Bảng 2. Hàm lấy giá trị trung bình độ đo $m(B)$

```
def run_averaged_experiment(n_repeats, m_samples):
    results = np.zeros(n_repeats)
    for i in range(n_repeats):
    n_B, _, _, _ = run_single_experiment(m_samples)
    results[i] = n_B
    return np.mean(results)
```

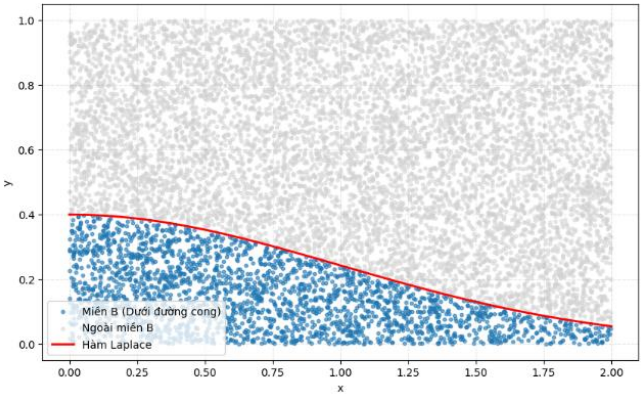
Ta xác định được $\overline{n_B} \approx 2386.88$. Vậy giá trị xấp xỉ trung bình độ đo $m(B)$ hoặc giá trị xấp xỉ trung bình của tích phân I là:

$$\overline{m(B)} = m(H) \times \frac{\overline{n_B}}{n_H} = 2 \times \frac{2386.55}{10000} = 0.4773$$

Thực hiện đối chiếu giá trị $m(B)$ và $\overline{m(B)}$ trong bảng tra hàm Laplace, ta thấy cả hai giá trị đều rất gần với giá trị 0.4772 với sai số sai số rất thấp:

$$\Delta m(B) = \left|0.4772 - \overline{m(B)}\right| = |0.4772 - 0.4773| = 0.0001$$

Dùng thư viện Matplotlib, ta có thể trực quan các điểm thoả mãn điều kiện (5) như sau:



Hình 1. Biểu diễn diện tích miền B bằng thư viện Matplotlib để tính xấp xỉ tích phân hàm Laplace

2) Bài toán 2

Giả sử ta cần tính xấp xỉ giá trị số π.

Trước tiên, ta chọn miền H là miền hình vuông sao cho

$$H = \{(x, y) : 0 < x, y < 1\} \quad (6)$$

Ta xác định được độ đo $m(H)$ là diện tích của hình vuông, tức $m(H) = 1 \times 1 = 1$.

Miền B là tập hợp các điểm nằm góc phần tư thứ nhất của hình tròn đơn vị, thoả mãn điều kiện:

$$B = \{(x, y) : x > 0, y > 0, x^2 + y^2 < 1\} \quad (7)$$

Đoạn chương trình sau thực hiện tạo sinh $n_H = 10000$ điểm toạ độ (x, y) đảm bảo tuân thủ luật phân phối đều nằm trong khoảng $[0,1]$:

Bảng 3. Hàm thực hiện tạo sinh n_H điểm (x, y) nằm trong khoảng $[0,1]$

```
def run_single_experiment(n_samples=10000):
    x = np.random.uniform(0, 1, n_samples)
    y = np.random.uniform(0, 1, n_samples)
    condition = x**2 + y**2 < 1
    n_inside = np.sum(condition)
    return n_inside, x, y, condition
```

Ta xác định được $n_B = 7874$, tức là có 7874 trên 10000 điểm n_H thoả điều kiện (7). Ta có:

$$\frac{n_B}{n_H} = \frac{m(B)}{m(H)} = \frac{S_B}{S_H} = \frac{\pi}{4} \Leftrightarrow \pi = 4 \times \frac{n_B}{n_H}$$

Do đó giá trị của π là $\pi = 4 \times \frac{7874}{10000} = 3.1496$

Để hạn chế sai số, ta thực hiện lặp lại 10000 lần và lấy giá trị trung bình của các mẫu bằng đoạn chương trình sau:

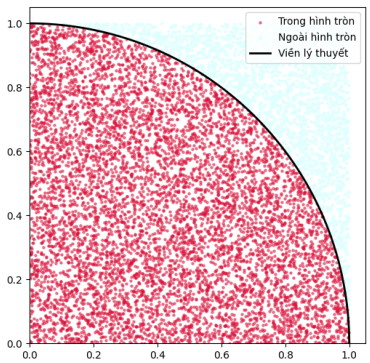
Bảng 4. Hàm lấy giá trị π trung bình

```
def run_averaged_experiment(n_repeats, m_samples):
    pi_values = np.zeros(n_repeats)
    count_values = np.zeros(n_repeats)
    for i in range(n_repeats):
        x = np.random.uniform(0, 1, m_samples)
        y = np.random.uniform(0, 1, m_samples)
        count = np.sum(x**2 + y**2 < 1)
        count_values[i] = count
        pi_values[i] = 4 * (count / m_samples)
    return np.mean(pi_values), np.mean(count_values)
```

Ta xác định được $\overline{n_B} \approx 7854.17$. Vậy giá trị xấp xỉ trung bình của số π là:

$$\overline{\pi} = 4 \times \frac{\overline{n_B}}{n_H} = 4 \times \frac{7854.17}{10000} = 3.14167$$

Dùng thư viện Matplotlib, ta có thể trực quan các điểm thoả mãn điều kiện (7) như sau:



Hình 2. Biểu diễn diện tích miền B bằng công cụ Matplotlib để tính xấp xỉ giá trị π

3) Bài toán 3

Giả sử chọn ngẫu nhiên 3 đoạn thẳng, mỗi đoạn thẳng có độ dài không vượt quá số $a > 0$ cho trước. Tìm xác suất sao cho các đoạn thẳng đó là các cạnh của một tam giác, của một tam giác tù.

Gọi x, y, z lần lượt là độ dài các đoạn thẳng. Gọi H là không gian mẫu sao miền H là một hình lập phương thoả mãn:

$$H = \{(x, y, z) : 0 \le x, y, z \le a\} \quad (8)$$

Ta xác định được độ đo $m(H)$ là thể tích của lập phương cạnh a, tức $m(H) = a^3$.

Để chuẩn hoá bài toán, ta chọn $a = 1$, tức $m(H) = 1$.

Gọi B và B' lần lượt là các miền chứa các đoạn thẳng cũng chính là các cạnh của tam giác và tam giác tù, thoả mãn các điều kiện sau:

$$B = \{(x, y, z) \in H : x + y > z, x + z > y, y + z > x\} \quad (9)$$

$$B' = \{(x, y, z) \in B : x^2 > y^2 + z^2 \cup y^2 > x^2 + z^2 \cup z^2 > x^2 + y^2\} \quad (10)$$

Đoạn chương trình sau thực hiện tạo sinh n_H điểm toạ độ (x, y, z) thuộc miền B với n_H lần lượt là 100, 1000, 10000, 100000 đảm bảo tuân thủ luật phân phối đều nằm trong khoảng $[0, a]$ (với $a = 1$) và thoả mãn điều kiện (9):

Bảng 5. Hàm thực hiện tạo sinh n_H điểm (x, y, z) thuộc miền B nằm trong khoảng $[0, a]$ với $a = 1$

```
def run_single_experiment(n_samples):
    data = np.random.uniform(0, 1, (n_samples, 3))
    x, y, z = data[:, 0], data[:, 1], data[:, 2]
    condition = (x + y > z) & (x + z > y) & (y + z > x)
    count = np.sum(condition)
    prob = count / n_samples
    return count, prob, data, condition
```

Về mặt lý thuyết hình học, xác suất để tạo thành một tam giác B thuộc miền H:

$$P(B) = \frac{m(B)}{m(H)} = \frac{\frac{a^3}{2}}{a^3} = \frac{1}{2} \quad (7)$$

Sử dụng phương pháp Monte Carlo, ta tìm được các giá trị n_B, $P(B)_{MC}$ tương ứng thông qua bảng mô tả:

Bảng 6. Kết quả trả về các giá trị n_B, $P(B)_{MC}$

n_H	100	1000	10000	100000
n_B	46	494	4979	50098
$P(B)_{MC}$	0.46	0.494	0.4979	0.5010
$\Delta P(B)$	0.0400	0.006	0.0021	0.0010

với sai số được tính bằng công thức $\Delta P(B) = \left| P(B) - P(B)_{MC} \right|$

Ta thấy các giá trị sai số trên rất nhỏ, không đáng kể. Tuy nhiên, để giảm thiểu các sai số, ta thực hiện lặp lại 10000 lần và lấy giá trị trung bình của n_H bằng đoạn chương trình sau:

Bảng 7. Hàm tính xác suất tạo thành tam giác tù với 10000 lần lặp

```
def run_averaged_experiment(n_repeats, n_samples):
    counts = np.zeros(n_repeats)
    for i in range(n_repeats):
        c, _, _, _ = run_single_experiment(n_samples)
        counts[i] = c
    avg_count = np.mean(counts)
    avg_prob = avg_count / n_samples
    return avg_count, avg_prob
```

Kết quả trả về các giá trị $\overline{n_B}$, $\overline{P(B)_{MC}}$ tương ứng được mô tả ở bảng sau đây:

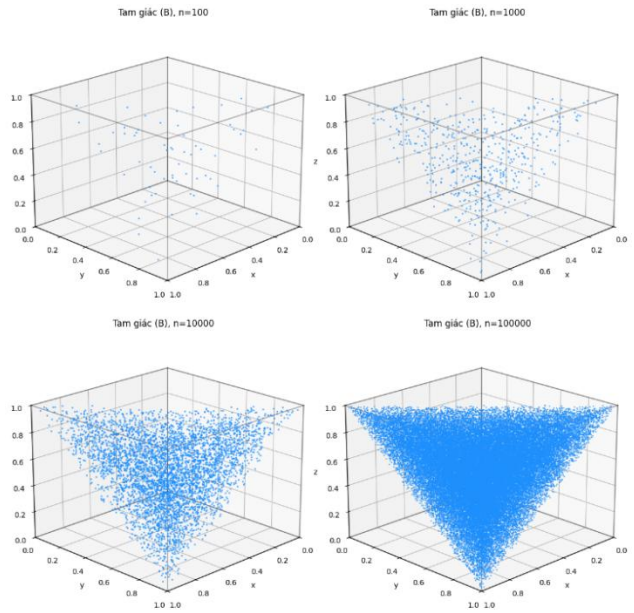
Bảng 8. Kết quả trả về các giá trị $\overline{n_B}$, $\overline{P(B)_{MC}}$

n_H	100	1000	10000	100000
$\overline{n_B}$	49.96	499.88	5000.28	49999.17
$\overline{P(B)_{MC}}$	0.4996	0.4999	0.5000	0.4999
$\Delta\overline{P(B)}$	0.0004	0.0001	0.0000	0.0001

với sai số được tính bằng công thức $\Delta\overline{P(B)} = \left|P(B) - \overline{P(B)_{MC}}\right|$

Kết quả trên hoàn toàn phù hợp với lý thuyết (7).

Dùng thư viện Matplotlib, ta có thể trực quan các điểm thoả mãn điều kiện (9) như sau:



Hình 3. Biểu diễn các điểm thuộc miền B tạo thành tam giác trong miền H bằng công cụ Matplotlib

Tuy nhiên, xét trường hợp các đoạn thẳng lấy ra là các cạnh của một tam giác tù, việc vẽ hình theo điều kiện thoả mãn không đơn giản.

Đoạn chương trình sau thực hiện tạo sinh n_H điểm toạ độ (x, y, z) thuộc miền B' với n_H lần lượt là 100, 1000, 10000, 100000 đảm bảo tuân thủ luật phân phối đều nằm trong khoảng $[0, a]$ (với $a = 1$) và thoả mãn điều kiện (10):

Bảng 9. Hàm thực hiện tạo sinh n_H điểm (x, y, z) thuộc miền B' nằm trong khoảng $[0, a]$ với $a = 1$

```
def run_single_experiment_obtuse(n_samples):
    data = np.random.uniform(0, 1, (n_samples, 3))
    x, y, z = data[:, 0], data[:, 1], data[:, 2]
    is_triangle = (x + y > z) & (x + z > y) & (y + z > x)
    is_obtuse_cond = (x**2 + y**2 < z**2) | \
                     (x**2 + z**2 < y**2) | \
                     (y**2 + z**2 < x**2)
    condition = is_triangle & is_obtuse_cond
    count = np.sum(condition)
    prob = count / n_samples
    return count, prob, data, condition
```

Về mặt lý thuyết hình học, xác suất để tạo thành một tam giác tù B' thuộc miền H:

$$P(B') = \frac{m(B')}{m(H)} = \frac{a^3 \times \frac{\pi - 2}{4}}{a^3} = \frac{\pi - 2}{4} \approx 0,2854 \quad (11)$$

Sử dụng phương pháp Monte Carlo, ta tìm được các giá trị $n_{B'}$, $P(B')_{MC}$ tương ứng thông qua bảng mô tả:

Bảng 10. Kết quả trả về các giá trị $n_{B'}$, $P(B')$

n_H	100	1000	10000	100000
$n_{B'}$	21	295	2812	28199
$P(B')_{MC}$	0.21	0.295	0.2812	0.2820
$\Delta P(B')$	0.0754	0.0096	0.0042	0.0034

với sai số được tính bằng công thức $\Delta\overline{P(B')} = \left|P(B') - \overline{P(B')_{MC}}\right|$

Để hạn chế sai số, ta thực hiện lặp lại 10000 lần và lấy các giá trị trung bình của n_H bằng đoạn chương trình sau:

Bảng 11. Hàm tính xác suất tạo thành tam giác tù với 10000 lần lặp

```
def run_averaged_experiment_obtuse(n_repeats, n_samples):
    counts = np.zeros(n_repeats)
    for i in range(n_repeats):
        c, _, _, _ = run_single_experiment_obtuse(n_samples)
        counts[i] = c
    avg_count = np.mean(counts)
    avg_prob = avg_count / n_samples
    return avg_count, avg_prob
```

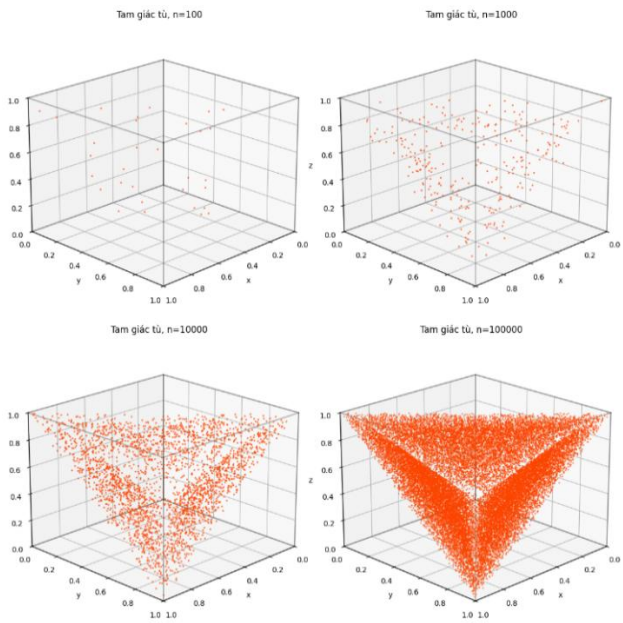
Kết quả trả về các giá trị $\overline{n_{B'}}$, $\overline{P(B')}$ tương ứng được mô tả ở bảng sau đây:

Bảng 12. Kết quả trả về các giá trị $\overline{n_{B'}}$, $\overline{P(B')}$

n_H	100	1000	10000	100000
$\overline{n_B}$	28.56	285.27	2853.76	28538.55
$\overline{P(B')}$	0.2856	0.2853	0.2854	0.2854
$\Delta\overline{P(B')}$	0.0002	0.0001	0.0000	0.0000

với sai số được tính bằng công thức $\Delta\overline{P(B')} = \left|P(B') - \overline{P(B')_{MC}}\right|$

Kết quả trên hoàn toàn phù hợp với lý thuyết (11)



Hình 4. Biểu diễn các điểm thuộc miền B' tạo thành tam giác trong miền H bằng công cụ Matplotlib

IV. KẾT LUẬN

Qua quá trình nghiên cứu lý thuyết và thực nghiệm mô phỏng trên ngôn ngữ Python, nhóm thực hiện đã đạt được các mục tiêu đề ra ban đầu về việc ứng dụng phương pháp Monte Carlo trong giải quyết các bài toán xác suất hình học. Thông qua kết quả thực nghiệm trên 03 bài toán, ta có thể thấy sự hội tụ rõ rệt của tần suất thực nghiệm về xác suất lý thuyết khi kích thước mẫu n_H tăng lên theo đúng Luật số lớn. Ngoài ra, việc sử dụng ngôn ngữ Python với các thư viện NumPy và Matplotlib đã mang lại hiệu quả cao trong tính toán và trực quan hóa.

Mặc dù phương pháp Monte Carlo rất mạnh mẽ trong việc giải quyết các bài toán có không gian mẫu phức tạp hoặc số chiều lớn, độ chính xác còn phụ thuộc chặt chẽ vào số lượng mẫu thử n_H vẫn là một ràng buộc đáng kể bằng cách tăng số lượng mẫu thử n_H. Trong tương lai, ta có thể áp dụng các kỹ thuật nâng cao như Mô phỏng Monte Carlo chuỗi Markov hoặc các phương pháp giảm phương sai (Variance Reduction) để đạt độ hội tụ nhanh hơn với số lượng mẫu ít hơn.

Tổng kết lại, mô phỏng Monte Carlo là một công cụ không thể thiếu trong Khoa học Dữ liệu, cho phép kiểm chứng các giả thuyết và tìm ra lời giải xấp xỉ cho các bài toán mà lời giải giải tích là bất khả thi hoặc quá tốn kém để thực hiện.

THAM KHẢO

[1] N. T. P. Đoan, "Ứng dụng của phương pháp mô phỏng Monte Carlo với xác suất hình học," 2025.

[2] S. H. Chan, Introduction to Probability for Data Science, 2021.

[3] Dương Tôn Đảm, Dương Tôn Thái Dương, Xác suất - Thống kê chuyên sâu, Đại học Quốc Gia TP.HCM, 2021.

[4] N. Q. Hỷ, Phương pháp mô phỏng số Monte Carlo, Nhà xuất bản Đại học Quốc gia Hà Nội., 2004.

[5] John Wiley, Sons, Uniform Distribution of Sequences, New York: Wiley-interscience Publication, 1974.

[6] Christian P. Robert, George Casella, Monte Carlo Statical Method (2nd Edition), New York: Springer-Verlag, 2004.

[7] "Pyplot tutorial," Matplotlib development team, [Online]. Available: https://matplotlib.org/. [Accessed 12 2025].

[8] "Pandas documentation," NumFOCUS, Inc, 25 9 2025. [Online]. Available: https://pandas.pydata.org/docs/. [Accessed 12 2025].